Machine Intelligence

2. State Space and Uninformed Search Methods

Daniel Fišer

AALBORG UNIVERSITY

Fall 2025

Agenda

1. State Space
2. How to Put Problem to Computer
3. Search Basics
4. Uniform-cost Search
5. Iterative Deepening

A Search Problem: Find a Route

Problem: Find a route from Copenhagen to Prague

- Starting in the initial state ... (Copenhagen)
- ... apply actions ... (move along road segments)
- ... to reach a goal state. (Prague)
- Preformance measure: Minimize summed-up action costs. (sum of road segment lengths)

A Search Problem: 15-puzzle

Problem: Move tiles to transform left state into right state.

9	2	12	6
5	7	14	13
3	4	1	11
15	10	8	

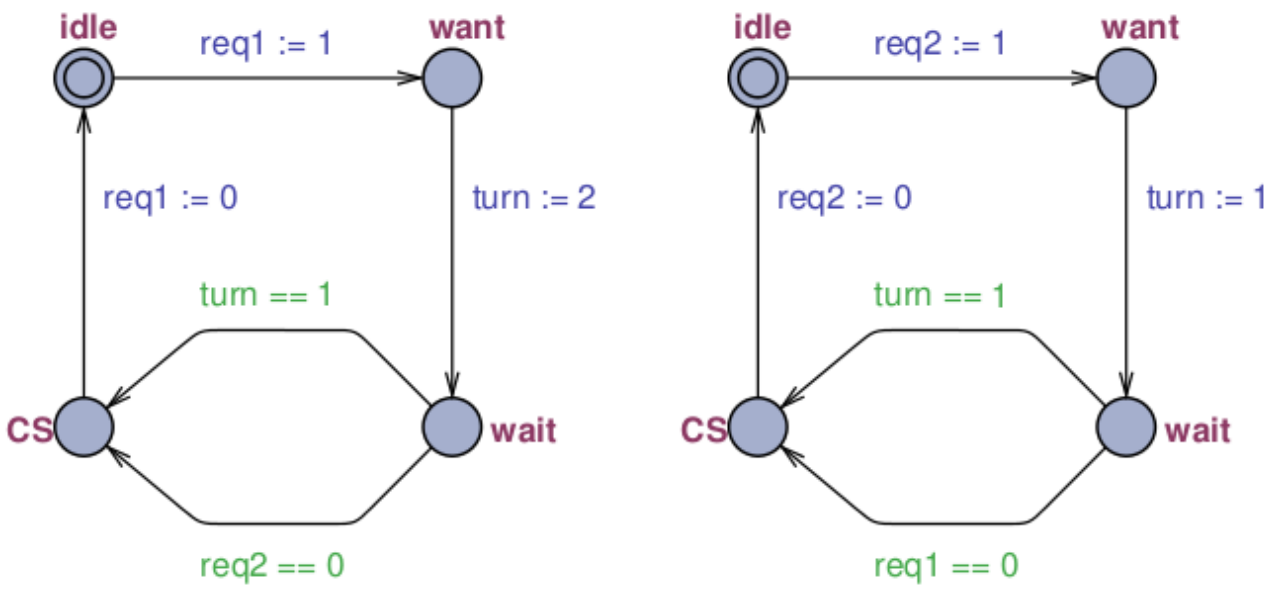
—

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

- Starting in the initial state ... (Left)
- ... apply actions ... (move tiles one by one)
- ... to reach a goal state. (Right)
- Preformance measure: Minimize summed-up action costs. (each move has cost 1, so we minimize number of moves)

A Search Problem: Finding SW Bugs

Problem: Finding bugs in software artifacts.



- Starting in the initial state ... (Both idle)
- ... apply actions ... (automaton transitions)
- ... to reach a goal state. (Goal=error—boths in critical section CS)
- Preformance measure: Minimize summed-up action costs. (each transition has cost 1, so we minimize the length of the error path)

State Space

Definition (State Space)

A state space is a tuple $\Theta = (\mathcal{S}, \mathcal{A}, \mathrm{cost}, \mathcal{T}, I, \mathcal{S}_G)$ where:

- $\mathcal{S}$ is a finite set of states.
- $\mathcal{A}$ is a finite set of actions.
- $\mathrm{cost} : \mathcal{A} \mapsto \mathbb{N}_0$ is the cost function.
- $\mathcal{T} \subseteq \mathcal{S} \times \mathcal{A} \times \mathcal{S}$ is the transition relation. We require that $\mathcal{T}$ is deterministic, i.e., for all $s \in \mathcal{S}$ and $a \in \mathcal{A}$, there is at most one state s' such that $(s, a, s') \in \mathcal{T}$. If such (s, a, s') exists, then we say that a is applicable in s.
- $I \in \mathcal{S}$ is the initial state
- $\mathcal{S}_G \subseteq \mathcal{S}$ is the set of goal states.

We say that Θ has the transition (s, a, s') if $(s, a, s') \in \mathcal{T}$.

Instead of (s, a, s'), we also write $s \xrightarrow{a} s'$, or $s \rightarrow s'$ when not interested in a.

We say that an action $a \in \mathcal{A}$ is unit cost if $\mathrm{cost}(a) = 1$, and we say that Θ is unit cost if, for all $a \in \mathcal{A}$, we have that $\mathrm{cost}(a) = 1$.

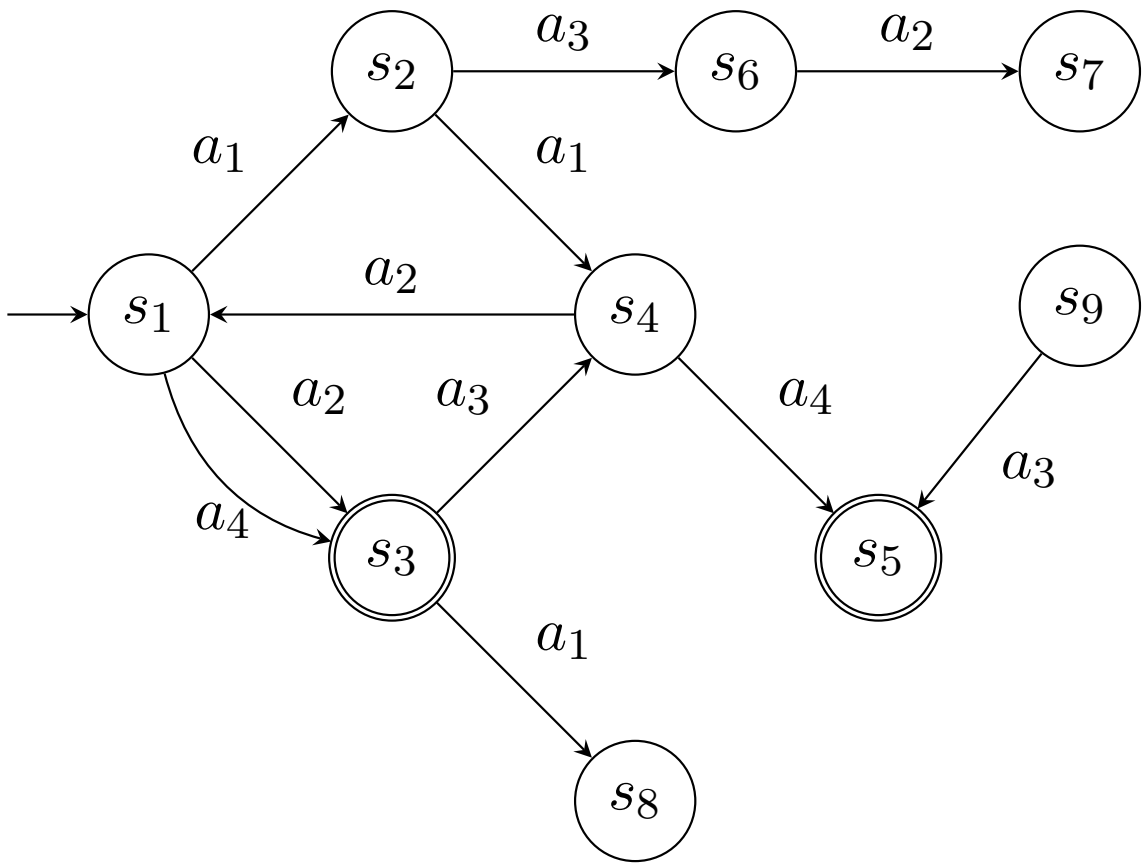
State Space

- s' is successor of s if $s \xrightarrow{a} s' \in \mathcal{T}$.
- s is predecessor of s' if $s \xrightarrow{a} s' \in \mathcal{T}$.
- s' is reachable from s if there exists a sequence of transitions $\tau = ((s_0, a_1, s_1), (s_1, a_2, s_2), \ldots, (s_{n-1}, a_n, s_n))$ s.t. $s = s_0$ and $s' = s_n$
  - we call τ a transition-path from s to s';
  - we call $\pi = (a_1, \ldots, a_n)$ an action-path from s to s';
  - we call $\sigma = (s_0, s_1, \ldots, s_n)$ a state-path from s to s';
  - when convenient we call transition-/action-/state-path simply path;
  - we can also write $\tau = s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} \ldots \xrightarrow{a_{n-1}} s_{n-1} \xrightarrow{a_n} s_n$
  - the cost of τ or π is $\mathrm{cost}(\tau) = \mathrm{cost}(\pi) = \sum_{i=1}^{n} \mathrm{cost}(a_i)$;
  - $n = 0$ possible, i.e., s is reachable from s (path from s to s is empty);
- s is reachable (without reference state) if it is reachable from I.
- s is solvable if some $s' \in \mathcal{S}_G$ is reachable from s, otherwise s is dead end.

Definition (State Space Solution)

Let $\Theta = (\mathcal{S}, \mathcal{A}, \mathrm{cost}, \mathcal{T}, I, \mathcal{S}_G)$ be a state space and let $s \in \mathcal{S}$ be a state. A solution for s is a path τ from s to some $s' \in \mathcal{S}_G$; τ is called optimal if $\mathrm{cost}(\tau)$ is minimal. A solution for I is called solution for Θ or simply solution. If a solution exists, we call Θ solvable, otherwise we call Θ unsolvable.

Illustration of a State Space



$\mathcal{S} = \{s_1, s_2, \ldots, s_9\}$
$\mathcal{A} = \{a_1, a_2, \ldots, a_4\}$
$I = s_1$
$\mathcal{S}_G = \{s_3, s_5\}$

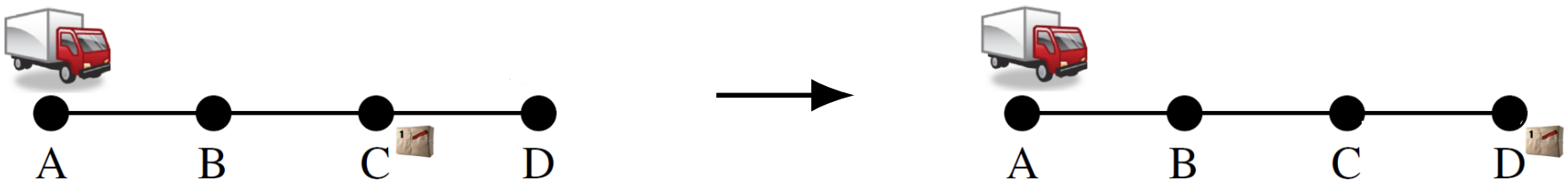
s_3 is a successor of s_1
s_4 is a predecessor of s_5
s_4 is reachable from s_1
s_5 is reachable
s_9 is not reachable
s_1 is solvable
s_6 is a dead end
...

Transition-path from s_1 to s_6: $(s_1, a_1, s_2), (s_2, a_3, s_6)$
Action-path from s_1 to s_6: (a_1, a_3)
State-path from s_1 to s_6: (s_1, s_2, s_6)

Exercise: Simple Logistics Problem

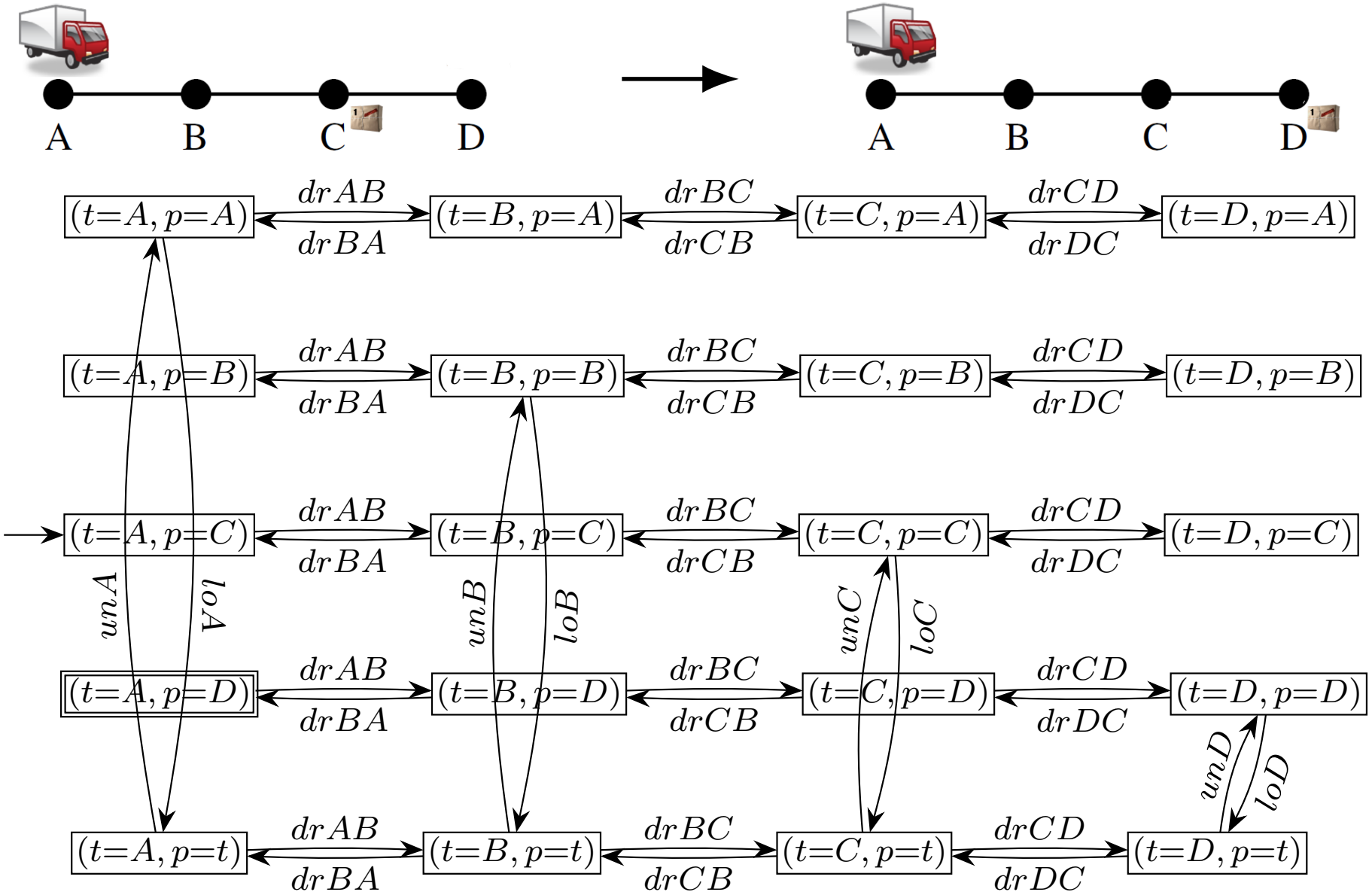
Simple logistics problem:

- Map of locations, one truck, one package
- Truck can move between connected locations, load/unload package
- The goal is: package at D, truck at A



- State representation:
  truck at one location: $t \in \{A, B, C, D\}$
  package at one location or loaded to truck: $p \in \{A, B, C, D, t\}$
- Initial state: $(t{=}A, p{=}C)$
- Goal state(s): $\{(t{=}A, p{=}D)\}$
- Actions:
  drive truck from X to Y if connected: $drAB, drBC, ...$
  load/unload package at X if both truck and package there:
  $loA, loB, ..., unA, unB, ...$

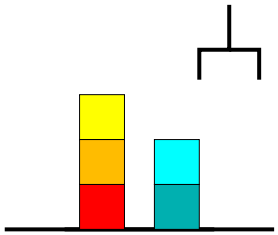
Exercise: Simple Logistics Problem – State Space



Exercise Sheet

Do exercises from Exercise Sheet 1: Section 1

How can we specify a problem Π?



Blocksworld problem:

- n blocks, 1 hand.
- A single action either takes a block with the hand or puts a block we're holding onto some other block/the table.

blocks	states
1	1
2	3
3	13
4	73
5	501
6	4051
7	37633
8	394353

blocks	states
9	4596553
10	58941091
11	824073141
12	12470162233
13	202976401213
14	3535017524403
15	65573803186921
16	1290434218669921

$\rightsquigarrow$ State spaces may be huge. In particular, the state space is typically exponentially large in the size of its specification via the problem Π (up next).

$\rightsquigarrow$ In other words: Search problems typically are computationallly hard (e.g., optimal Blocksworld solving is NP-complete).

$\rightsquigarrow$ So the question becomes: What are suitable problem representations?

Option 1: Explicit Representation

$\rightsquigarrow$ The explicit representation describes Π simply in terms of its state space:

Explicit Description of a Problem

$\Pi = \Theta$: We simply input the state space (e.g., as a graph).

- "Specifying the problem" = writing down the entire state space.
- Impossible for large state spaces.
- Can be solved easily **in the size of the state space**: Dijkstra's algorithm.

$\rightsquigarrow$ Explicit descriptions do not have the ability to compactly describe large state spaces.

$\rightsquigarrow$ They are used if state spaces are "small" (only 100000s of states) and runtime is very limited. This is typically the case in route planning. A prominent application is in computer games, where routes for all non-player agents must be computed in microseconds.

Option 2: Black-box Representation

$\rightsquigarrow$ The blackbox description of a problem Π is an API (a programming interface) providing functionality allowing to construct the state space:

Blackbox Description of a Problem

- InitialState(): Returns the initial state of the problem.
- GoalTest(s): Returns a Boolean, "true" iff state s is a goal state.
- Cost(a): Returns the cost of action a.
- Actions(s): Returns the set of actions that are applicable to state s.
- Successor(s, a): Requires that action a is applicable to state s, i.e., there is a transition $s \xrightarrow{a} s'$. Returns the outcome state s'.

- "Specifying the problem" = programming the API.
- Huge state spaces can be specified with little program code.

$\rightsquigarrow$ The API does not provide the search with any knowledge about the problem, other than the bare essentials needed to generate the state space. Hence the name "blackbox", as opposed to: up next.

Option 3: Declarative/White-box Representation

$\rightsquigarrow$ The declarative representation (model) of Π comes in form of a language:

Declarative Description of a Problem

There are many ways to do this. Here's one:

- $\mathcal{V}$: Set of variables
- I: Assignment to all variables $\mathcal{V}$, defining the initial state.
- G: Partial assignment to some variables $\mathcal{V}$, defining goal states.
- $\mathcal{A}$: Set of actions $a \in \mathcal{A}$, each with precondition pre_a and effect eff_a, both partial assignments to $\mathcal{V}$; a applicable to s if pre_a agrees with s, outcome of applying a in s is s changed according to eff_a.
- cost: Maps each $a \in \mathcal{A}$ to its cost $\mathrm{cost}(a)$.

- This language is called "FDR"; we'll get back to it in Lecture 4.
- "Specifying the problem" = writing FDR. The computer then inputs that description and can generate the state space.

$\rightsquigarrow$ Declarative representations are strictly more powerful than blackbox ones. They allow to implement the API, and much more (e.g., analyze/simplify the problem).

Declarative Representations Allow General Solvers

$\rightsquigarrow$ Declarative representations enable general problem solving:

some new (search) problem

$\downarrow$

describe the problem in generic language

$\downarrow$

use off-the-shelf solver

$\downarrow$

solution to the problem

- Little programming effort, easy to adapt to changes.
- Core topic of my research: So, come speak to me if you find it interesting and want to work on something in this direction.
- In this and the next lecture, we assume the blackbox description. Explicit descriptions will only be used in (some) illustrative examples.
- In principle, the search algorithms we will discuss can be used with *any* problem representation that allows to implement the blackbox API.

Questionnaire

Question

What kind of description do you use when explaining your problems to somebody else?

(A): Blackbox

(B): Declarative

(C): Explicit

(D): I don't have problems

$\rightsquigarrow$ (A), (C): Presumably, you are not crazy enough to do that.

$\rightsquigarrow$ (B): Natural language is (amongst many other things) a kind of problem description language, so this answer makes most sense (to me). Example: Explaining to somebody the rules of "Missionaries and Cannibals".

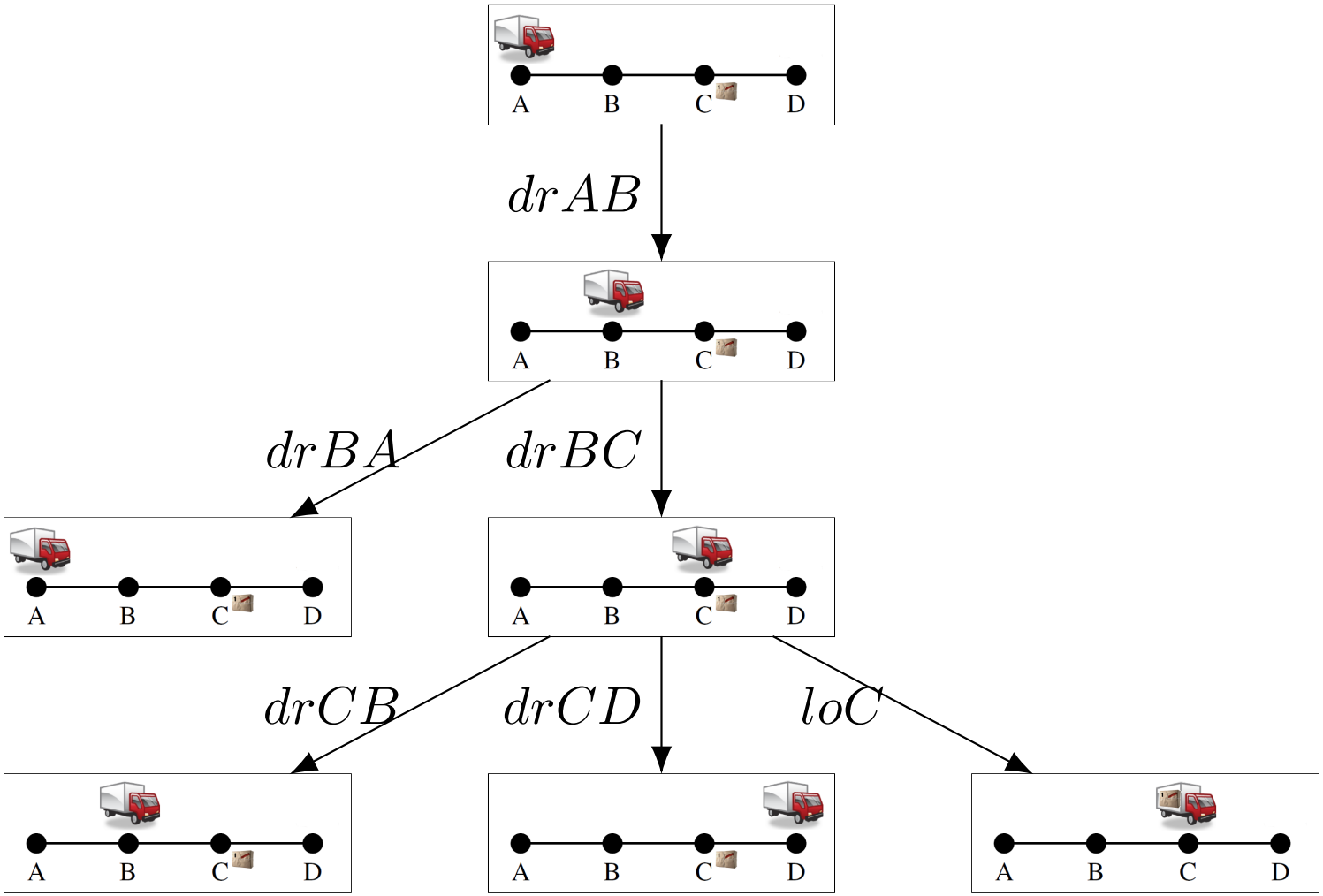
$\rightsquigarrow$ (D): Actually that answer is reasonable given the limited notion of "problem" (= search problem) we are considering here.

Exercise Sheet

Do exercises from Exercise Sheet 1: Section 2

Search Illustration

How to "search"? Start at the initial state. Then, step-by-step, expand a state by generating its successors ...



Search Terminology

Search node n: Contains a *state* reached by the search, plus information about how it was reached.

Path cost $g(n)$: The cost of the path reaching n.

Optimal cost g^*: The cost of an optimal solution path. For a state s, $g^*(s)$ is the cost of a cheapest path reaching s.

Node expansion: Generating all successors of a node, by applying all actions applicable to the node's state s. Afterwards, the *state* s itself is also said to be expanded.

Open list: Set of all *nodes* that currently are candidates for expansion. Also called frontier.

Closed list: Set of all *states* that were already expanded. Used only in graph search, not in tree search (up next). Also called explored set.

Tree Search vs. Graph Search

Duplicate Elimination:

- Maintain a closed list.
- Check for each generated state s' whether s' is in the closed list. If so, discard s'.

Tree Search:

- ... is another word for "don't use duplicate elimination".
- Search space is "tree-like": We do not consider the possibility that the same state may be reached from more than one predecessor.
- The same state may appear in many search nodes.
- Main advantage: lower memory consumption (no closed list needed).

Graph Search:

- ... is another word for "use duplicate elimination".
- Search space is "graph-like": We do consider said possibility.

Criteria for Evaluating Search Strategies

Guarantees:

Completeness: Is the strategy **guaranteed** to find a solution when there is one?

Optimality: Are the returned solutions **guaranteed** to be optimal?

Complexity:

Time Complexity: How long does it take to find a solution? (Measured in generated states.)

Space Complexity: How much memory does the search require? (Measured in states that we hold in memory.)

Typical state space features governing complexity:

Branching factor b: How many successors does each state have?

Goal depth d: The number of actions required to reach the shallowest goal state.

Maximum depth m: The maximum depth of the search tree.

Blind vs. Informed Search

Blind search vs. informed search:

- Blind search does not require any input beyond the API.
  - Pros: No additional work for the programmer.
  - Cons: It's not called "blind" for nothing ... same expansion order regardless what the problem actually is. Rarely effective in practice.
- Informed search requires as additional input: a heuristic function h (next lecture) that maps states to estimates of their goal distance.
  - Pros: Typically more effective in practice.
  - Cons: Somebody's gotta come up with/implement h.

$\rightsquigarrow$ Note: In Automated Planning, h is generated automatically from the declarative problem description (Lectures 4 and 5).

Today: Blind Search

Blind search strategies covered:

- Breadth-first search, Depth-first search. You already know them from Graph theory, so we'll go quickly over it.
- Uniform-cost search. Optimal for non-unit costs.
- Iterative deepening search. Combines advantages of breadth-first search and depth-first search.

Blind search strategy not covered:

- Bi-directional search. Two separate search spaces, one forward from the initial state, the other backward from the goal. Stops when the two search spaces overlap.

First, let's wrap our API

```
function InitNode() returns a search node
    return a new search node n such that
        n.State = InitState(), /* State stored in this search node */
        n.Pred = null, /* Predecessor (parent) node */
        n.Action = null, /* Achiever action */
        g(n) = 0 /* the cost of the path from the initial state to n */
function SuccNode(search node p, action a) returns a set of search nodes
    return a new search node n such that
        n.State = Successor(p.State, a), /* Store successor state of p.State */
        n.Pred = p, /* Expanding p, so p is the predecessor */
        n.Action = a, /* i.e., we record transition p.State -a-> n.State */
        g(n) = g(p)+Cost(a) /* Accum. the cost of reaching p plus cost(a) */
function ExtractSolution(search node n) returns an action-path π
    π ← ();
    while n.Pred ≠ null do
        π ← n.Action ∘ π /* Prepend n.Action before π */
        n ← n.Pred /* Step back to the recorded predecessor */
    return π
```

⇝ Search node records extra information useful when searching state space.

⇝ We could use n.g/n.CostOfPath/... instead of $g(n)$, but $g(n)$ is more consistent with what will come next.

Depth-first Search (DFS)

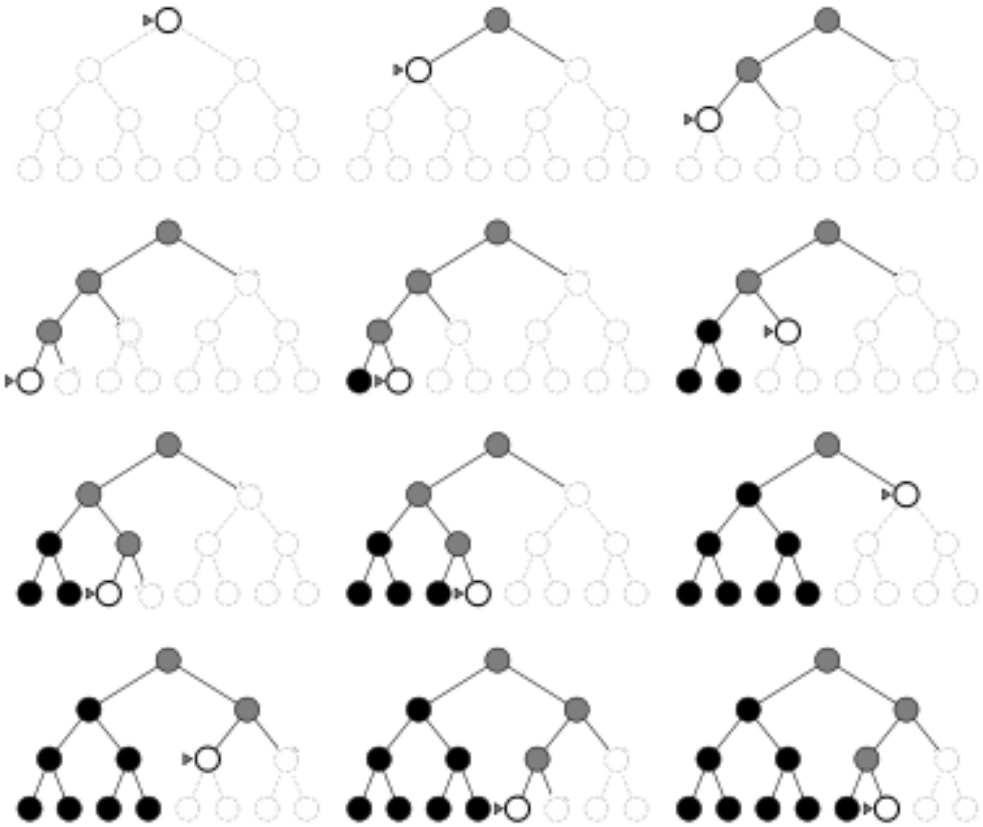
```
function DFS() returns a solution, or "unsolvable"
    n ← InitNode();
    Q ← Empty LIFO queue (stack);
    Push(Q,n) /* Push n into the queue Q                    */
    while Q is not empty do
        n ← Pop(Q) /* Pop next search node from Q           */
        if GoalTest(n.State) then return ExtractSolution(n);
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n,a);
            Push(Q,n′);
    return "unsolvable"
```

$\rightsquigarrow$ DFS = go deep and backtrack when there is nothing to expand.

Depth-first Search (DFS): Illustration

$\rightsquigarrow$ DFS = go deep and backtrack when there is nothing to expand.

Illustration: (Nodes at depth 3 are assumed to have no successors)



Depth-first Search (DFS)

```
function DFSv2() returns a solution, or "unsolvable"
    n ← InitNode();
    Q ← Empty LIFO queue (stack);
    Push(Q,n) /* Push n into the queue Q                    */
    closed ← ∅;
    while Q is not empty do
        n ← Pop(Q) /* Pop next search node from Q           */
        if GoalTest(n.State) then return ExtractSolution(n);
        closed ← closed ∪ {n.State};
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n,a);
            if n′.State ∉ closed then Push(Q,n′);
    return "unsolvable"
```

⇝ DFS = go deep and backtrack when there is nothing to expand.

⇝ This variant of DFS is a tree search.

⇝ Can we turn it into graph search by implementing duplicate elimination?

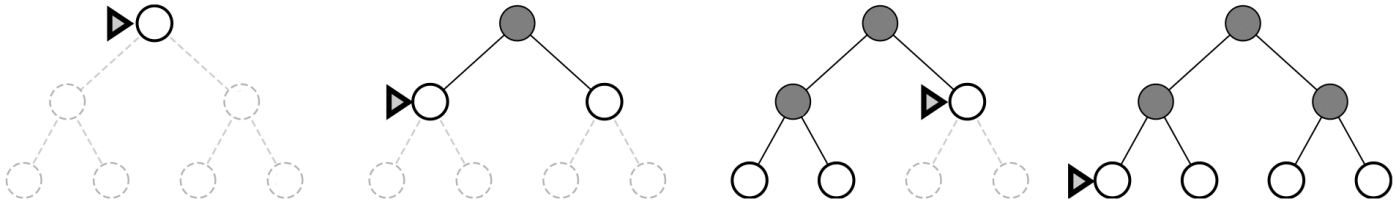
Complete?	Optimal?	Time complexity	Space complexity
DFS:No, DFSv2:Yes	No	$O(b^m)$	DFS:$O(bm)$, DFSv2:$O(b^m)$

Breath-first Search (BFS)

```
function BFS() returns a solution, or "unsolvable"
    n ← InitNode();
    Q ← Empty FIFO queue;
    Push(Q,n);
    while Q is not empty do
        n ← Pop(Q);
        if GoalTest(n.State) then return ExtractSolution(n);
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n,a);
            Push(Q,n′);
    return "unsolvable"
```

⇝ BFS = go wide, one layer at the time.

⇝ BFS is a tree search.



Complete?	Optimal?	Time complexity	Space complexity
Yes	Only for unit cost	$O(b^d)$	$O(b^d)$

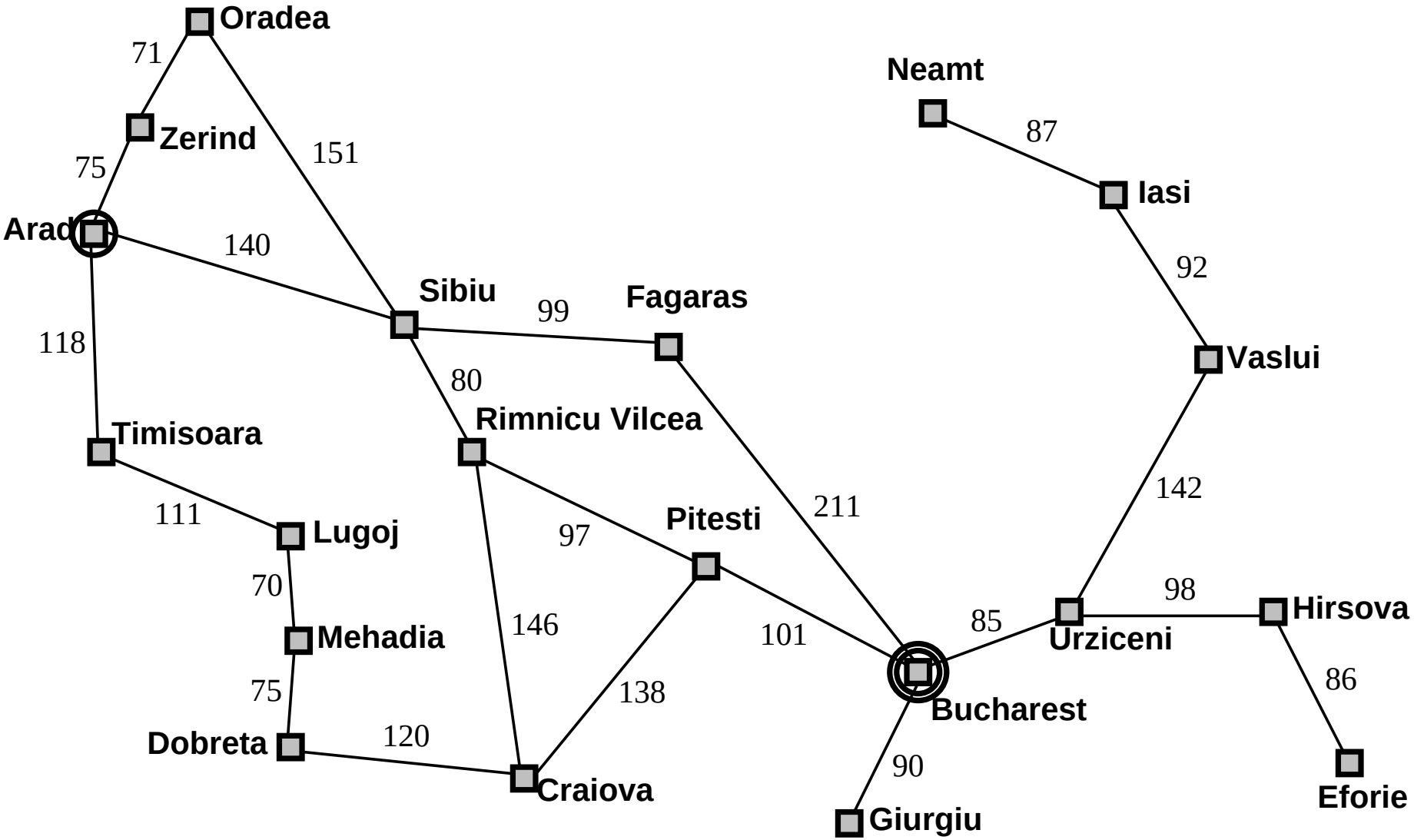
Exercise Sheet

Do exercises from Exercise Sheet 1: Section 3

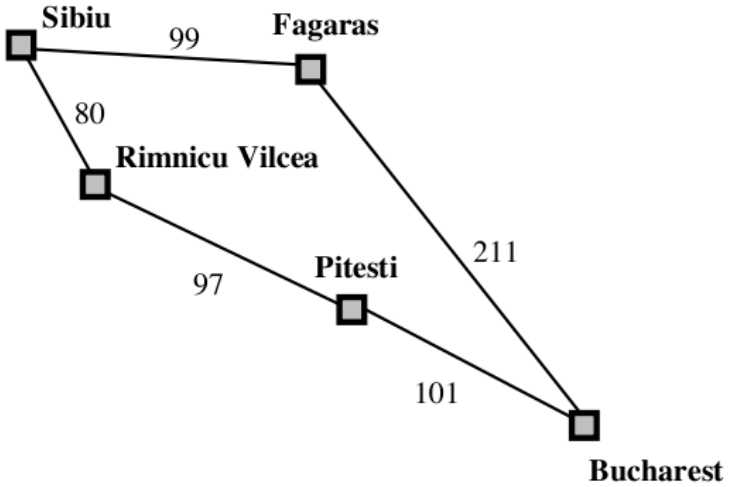
Uniform-cost Search

```
function UniformCostSearch() returns a solution, or "unsolvable"
    n ← InitNode();
    open ← Empty queue of search nodes ordered by ascending g-value;
    /* Let States(open) be a set of all n.State from open                    */
    closed ← ∅ /* Set of already processed states                            */
    Insert n into open;
    while open is not empty do
        n ← Pop(open) /* Pop search node n with lowest g(n)                  */
        if GoalTest(n.State) then return ExtractSolution(n);
        closed ← closed ∪ n.State;
        for each a ∈ Actions(n.State) do
            n′ ← SuccNode(n,a) /* Recall that g(n′) = g(n) + cost(a)        */
            if n′.State ∉ closed and n′.State ∉ States(open) then
                /* Insert new / unseen states                                */
                Insert n′ into open;
            else if n″ ∈ open s.t. n″.State = n′.State and g(n′) < g(n″) then
                /* We found a cheaper path to already seen state             */
                Replace n″ in open with n′;
    return "unsolvable"
```

Russel & Norvig's Example: Route Planning in Romania



Route Planning in Romania: Uniform-Cost Search



Search protocol:

1. Expand Sibiu, generating Rimnicu $g = 80$, Fagaras $g = 99$.
2. Expand Rimnicu, generating Pitesti $g = 80 + 97 = 177$ (as well as Sibiu which is already explored and thus pruned).
3. Expand Fagaras, generating Bucharest $g = 99 + 211 = 310$.
4. Expand Pitesti, generating Bucharest $g = 177 + 101 = 278$;
   Replace Bucharest $g = 310$ with Bucharest $g = 278$ in open!
5. Expand Bucharest $g = 278$.

Uniform-Cost Search: Guarantees and Complexity

Important observation: Uniform-cost search is equivalent to Dijkstra's algorithm on the state space graph (obvious from the definition of the two algorithms).

The only differences are:
(a) we generate only a part of that graph incrementally, whereas Dijkstra inputs and processes the whole graph;
(b) we stop when we reach any goal state (rather than a fixed target state given in the input).

Therefore, **Uniform-cost search is optimal.**

- **Completeness**: Yes, thanks to duplicate elimination—in the worst case, we explore the whole state space (and don't get stuck in cycles).
- **Time complexity**: $O(b^{1+\lfloor g^*/\epsilon \rfloor})$ where g^* denotes the cost of an optimal solution, and ϵ is the positive cost of the cheapest action.

  This is because we need explore all states with $g(s) < g^*$, and in the worst-case scenario also $g(s) = g^*$.
- **Space complexity**: Same as time complexity (because of the closed list).

Exercise Sheet

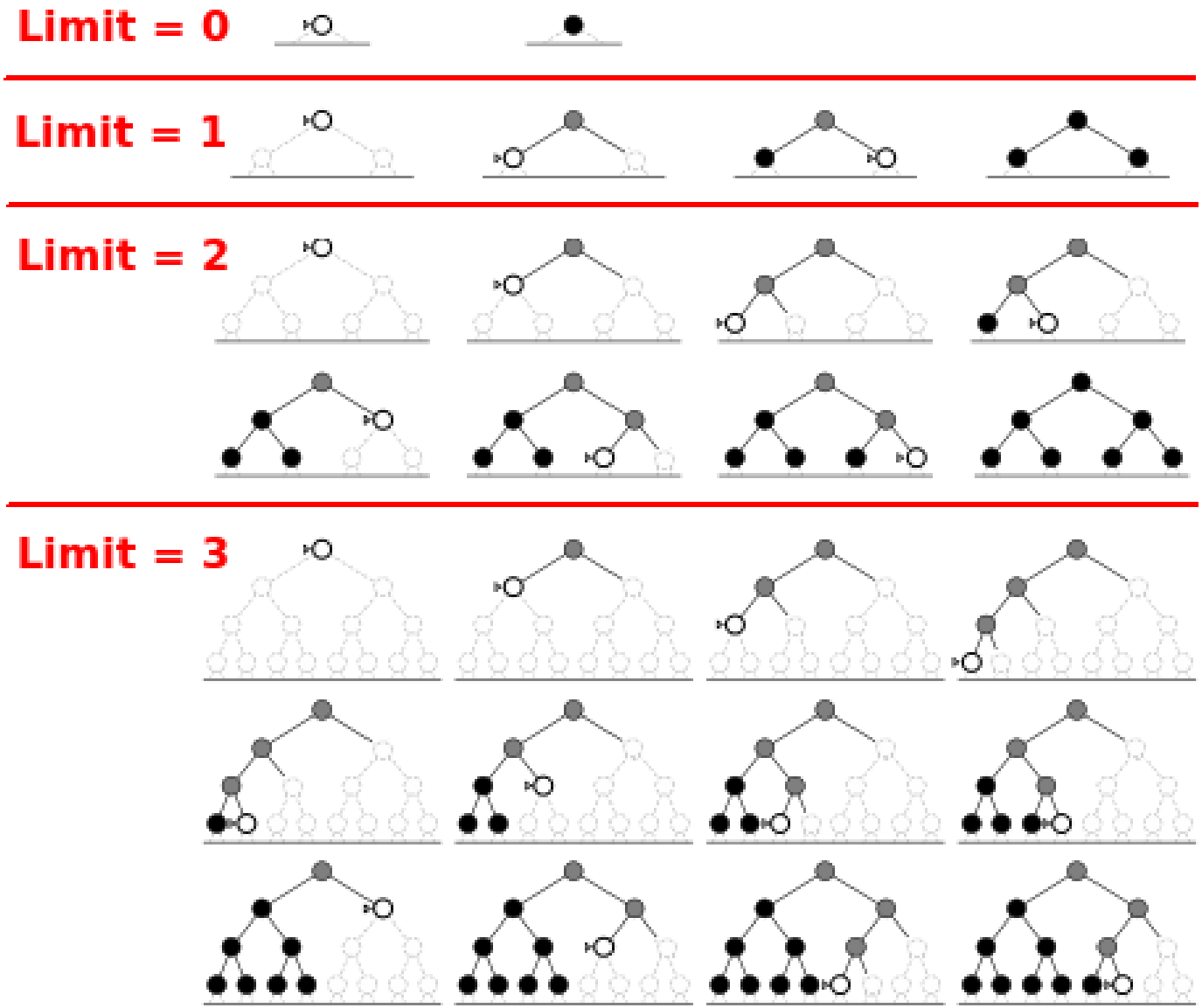
Do exercises from Exercise Sheet 1: Section 4

Iterative Deepening

```
function IterativeDeepening() returns a solution, or "unsolvable"
    n ← InitNode();
    for depth = 0..∞ do
        result ← DFSLimited(n, depth);
        if result ≠ "cutoff" then return results;
function DFSLimited(n,depth) returns a solution, "unsolvable", or "cutoff"
    if GoalTest(n.State) then return ExtractSolution(n);
    if GoalTest(depth = 0) then return "cutoff";
    cutoffOccured ← false;
    for each a ∈ Actions(n.State) do
        n′ ← SuccNode(n,a);
        result ← DFSLimited(n′, depth − 1);
        if result = "cutoff" then cutoffOccured ← true;
        else if result ≠ "unsolvable" then return result;
    if cutoffOccured = true then return "cutoff";
    else return "unsolvable";
```

⇝ Iterative Deepening = Run DFS only to a limited depth, increase the limit step by step.

Iterative Deepening Search: Illustration



Iterative Deepening Search: Guarantees and Complexity

Completeness: Yes! Limited depth for `DFSLimited` means that it cannot be stuck in a cycle forever, and for each depth limit L, we explore the whole part of the state space up to this limit in the worst case.
(Nevertheless: What will happen if there is no solution and the state space has cycles?)

Optimality: Only for unit cost. We explore all states at depth L before we check any state at depth $L+1$, i.e., Iterative Deepening behaves similarly as BFS in this aspect and we find the shallowest goal first.

Time complexity: $O(b^d)$

Breadth-First-Search	$b + b^2 + \cdots + b^{d-1} + b^d \in O(b^d)$
Iterative Deepening Search	$(d)b + (d-1)b^2 + \cdots + 3b^{d-2} + 2b^{d-1} + 1b^d \in O(b^d)$

Example: $b = 10, d = 5$

Breadth-First Search	$10 + 100 + 1,000 + 10,000 + 100,000 = 111,110$
Iterative Deepening Search	$50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$

Space complexity: $O(bd)$ – we clean memory after each cycle, so the most we need is for the final depth limit, and this is a single run of DFS.

Exercise Sheet

Do exercises from Exercise Sheet 1: Section 5

Summary of Blind Search Methods

	Complete?	Optimal?	Time complexity	Space complexity
DFS	No	No	$O(b^m)$	$O(bm)$
BFS	Yes	Only for unit cost	$O(b^d)$	$O(b^d)$
UCS	Yes	Yes	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$	$O(b^{1+\lfloor g^*/\epsilon \rfloor})$
IDS	Yes	Only for unit cost	$O(b^d)$	$O(bd)$

- DFS = Depth-first Search
- BFS = Breadth-first Search
- UCS = Uniform-cost Search
- IDS = Iterative Deepening
- b = branching factor
- d = minimum goal depth
- m = maximum search depth
- g^* = cost of the optimal solution
- ϵ = minimum positive action cost